Lab 15 — Opslag en StatefulSets

Theorie — hoe een cluster dat zijn Pods weggooit toch een database bewaart: volumes die overleven, een opslagklasse die ze maakt, en een controller die de Pods een naam en een schijf geeft die ze terugvinden.

Doelstellingen

- De keten **PersistentVolume → PersistentVolumeClaim → Pod** begrijpen, en de rol van de **StorageClass**.
- De toegangsmodi en het terugwinbeleid lezen, en weten wat `local-path` doet op k3s.
- Een **Deployment** onderscheiden van een **StatefulSet**: stabiele identiteit, één volume per Pod, *headless* Service.
- PostgreSQL uitrollen als StatefulSet en nagaan dat de gegevens het verwijderen van de Pod overleven.
- Weten hoe je back-upt, en wanneer een database niets in het cluster te zoeken heeft.

---

1. Het probleem: wegwerpbare Pods, gegevens die dat niet zijn

In lab 06 was de regel: een gegeven dat moet blijven, zit niet *in* de container. In het cluster is het erger: een Pod is wegwerpbaar (lab 11), wordt bij elke rollout vervangen (lab 12), en kan op **een andere node** terechtkomen waar de lokale schijf van de vorige niet bestaat. Een `emptyDir` (een volume gebonden aan het leven van de Pod) of een `hostPath` (een map van de node, lab 11) beantwoorden die behoefte niet. Er is opslag nodig die **onafhankelijk** van de Pod en van de node bestaat, en die het cluster aan de Pod kan koppelen waar die ook staat.

> **→ DOCKER-HERHALING**
> Het named volume van lab 06 (`podman volume create pgdata`) wordt een **PersistentVolume**; de `-v pgdata:/var/lib/postgresql/data` wordt een **PersistentVolumeClaim** waarnaar het template verwijst; en `podman volume export` wordt… hetzelfde, een `pg_dump` of een kopie van het volume. Wat verandert: het volume kan een netwerkschijf zijn (EBS, Azure Disk, NFS, Ceph) die een driver aan de node koppelt, en het cluster beheert de levenscyclus.

2. Drie objecten voor één schijf

Object	Wie het schrijft	Wat het zegt
StorageClass	het platformteam (of de distributie)	"zo maak je een volume": de driver (*provisioner*), zijn parameters, het terugwinbeleid, het moment van aanmaak
PersistentVolumeClaim (PVC)	**jij**, in de namespace van de applicatie	"ik wil 500 Mi, lezen-schrijven, van de standaardklasse"
PersistentVolume (PV)	de *provisioner*, automatisch	"hier is een volume van 500 Mi, op deze schijf, gekoppeld aan die PVC"

De Pod kent alleen de **PVC** (`persistentVolumeClaim: {claimName: demo-data}`). De StorageClass zet de vraag om in een echt volume: dat is **dynamische provisionering**, die het tijdperk verving

waarin een beheerder de PV's met de hand aanmaakte. `kubectl get pvc` toont `Pending` zolang er geen volume gekoppeld is, dan `Bound` met de naam van de PV (`pvc-<uuid>`).

Twee instellingen van de StorageClass tellen dagelijks. **`volumeBindingMode: WaitForFirstConsumer`**: het volume wordt pas aangemaakt wanneer een Pod de PVC gebruikt — zodat het op de juiste node kan komen; een PVC alleen blijft dus `Pending`, en dat is geen fout. **`reclaimPolicy: Delete`**: de PVC verwijderen verwijdert de PV **en zijn gegevens**; `Retain` houdt de PV verweesd, met de hand op te ruimen. De **toegangsmodi** zeggen wie mag mounten: `ReadWriteOnce` (één node tegelijk — een schijf), `ReadOnlyMany`, `ReadWriteMany` (meerdere nodes — NFS, een netwerkbestandssysteem).

K3S
De standaard-StorageClass van k3s, **`local-path`**, maakt een gewone map op de node (`/var/lib/rancher/k3s/storage/pvc-<uuid>_<namespace>_<pvc>`) en markeert de PV met een **node-affiniteit**: de Pod kan nooit elders gepland worden. Op één node is dat onzichtbaar; op een echt cluster is het precies wat je voor een database niet wilt, en gebruik je een **CSI**-driver (*Container Storage Interface*) naar netwerkopslag: `gp3` op EKS, `managed-csi` op AKS, Longhorn of Ceph op kale machines. Het manifest van de PVC verandert niet; de StorageClass wel.

→ LINUX
Onder de PVC zit altijd een **mount**: de kubelet koppelt de schijf (of de map) aan de node, formatteert ze indien nodig (`ext4` standaard), en mount ze in de container zoals een bind mount van lab 06 — met dezelfde UID-vragen: PostgreSQL schrijft als UID 70, en `fsGroup` in de `securityContext` van de Pod dient om het volume aan de juiste groep te geven wanneer de driver dat niet doet.

3. Deployment of StatefulSet

Een Deployment met een PVC werkt voor **één** instantie van een applicatie die het verdraagt een Pod te verliezen en er een andere te krijgen, op voorwaarde dat de twee nooit tegelijk op dezelfde schijf draaien. Voor een database ontbreken hem twee dingen:

- **Een stabiele identiteit.** De Pods van een Deployment heten `api-6c7bb54fc5-2th65`; die van een **StatefulSet** heten `db-0`, `db-1`, `db-2`, in volgorde, en een opnieuw aangemaakte Pod **neemt zijn naam terug**.
- **Eén volume per Pod.** Met `volumeClaimTemplates` krijgt elke Pod **zijn** PVC, genoemd `data-db-0`, `data-db-1`…, die bij elke nieuwe aanmaak aan hem gekoppeld wordt. Een Deployment met drie replica's zou dezelfde PVC delen (onmogelijk in `ReadWriteOnce`, rampzalig voor een database).

Daar komen een **geordende start en stop** (`db-0` vóór `db-1`), een update **één Pod tegelijk** in omgekeerde volgorde, en het **headless Service** (`clusterIP: None`) bij: in plaats van een virtueel IP dat verdeelt, geeft de DNS rechtstreeks de IP's van de Pods terug, en elke Pod heeft zijn naam, `db-0.db.lab15.svc.cluster.local`. Dat laat een replica `db-1` weten dat haar primaire `db-0` is.

ONTHOUDEN
Een StatefulSet verwijderen **verwijdert zijn PVC's niet**: de gegevens blijven, en een nieuwe StatefulSet met dezelfde naam vindt ze terug. Terugschalen naar `replicas: 0` evenmin. Het is het symmetrische vangnet van `podman compose down` uit lab 09 — en daarom neemt een verwijderde namespace de PVC's mee (en, met `Delete`, de gegevens): lees `kubectl get pvc` vooraf.

> **→ POSTGRESQL**
> De officiële image wil een `PGDATA` die **leeg of al geïnitialiseerd** is, en `lost+found` (aangemaakt door `ext4` in de root van een schijf) doet ze falen: vandaar de submap `PGDATA=/var/lib/postgresql/data/pgdata` in de StatefulSet. En een StatefulSet met `replicas: 3` maakt **geen** drie PostgreSQL-replica's: hij maakt drie onafhankelijke databases. Replicatie is het werk van een **operator** (CloudNativePG, Zalando, Crunchy) die primaire, replica's, failover en back-ups beheert — die rol je uit in productie, geen kale StatefulSet.

4. Back-uppen, alweer

De regel van lab 06 verandert niet: **logisch** met `kubectl exec db-0 -- pg_dump -U postgres > dump.sql` (consistent terwijl de database draait), **fysiek** via een *snapshot* van het volume (`VolumeSnapshot`, als de CSI-driver het toelaat — `local-path` niet) of een kopie met gestopte Pod. Een operator automatiseert beide naar objectopslag. En de vraag "waar zit het gegeven?" heeft een antwoord in twee commando's: `kubectl get pvc` (welke PVC), `kubectl get pv <naam> -o yaml` (welke schijf, welke node, welk beleid).

5. In het bedrijf

- **De database zit vaak niet in het cluster**: een beheerde dienst (RDS, Cloud SQL, Azure Database) biedt back-ups, replicatie en updates zonder dat je ernaar omkijkt; de applicatie bereikt ze via een Service `ExternalName` of een ConfigMap met de URL. Dat is de standaardkeuze van teams die PostgreSQL niet zelf willen uitbaten.
- **In het cluster: een operator**, nooit een met de hand geschreven StatefulSet, en een netwerk-StorageClass met snapshots.
- **"Zelfgemaakte" StatefulSets** blijven legitiem voor testomgevingen, caches, wachtrijen, en om te begrijpen wat de operator in jouw plaats doet.
- **De PVC is de grens**: de applicatie vraagt ruimte, het platform beslist waar ze vandaan komt.

Onthouden

- Pod → PVC → PV ← StorageClass: de applicatie vraagt, de klasse maakt, het cluster koppelt.
- `WaitForFirstConsumer` = een PVC alleen blijft `Pending`; `Delete` = de PVC verwijderen verwijdert de gegevens.
- `local-path` bij k3s = een map op de node met affiniteit; in productie een CSI-netwerkdriver.
- StatefulSet = stabiele namen (`db-0`), één PVC per Pod (`data-db-0`), geordende start, headless Service (`db-0.db`).
- De StatefulSet verwijderen of naar nul terugschalen behoudt de PVC's; de namespace verwijderen neemt ze mee.
- Drie replica's van een PostgreSQL-StatefulSet = drie databases; replicatie is het werk van een operator.
- Back-up: `pg_dump` via `kubectl exec`, CSI-snapshots, of een operator — en vaak een beheerde database buiten het cluster.

Woordenschat

PersistentVolume (PV): een echt volume, object van het cluster. — **PersistentVolumeClaim (PVC)**: een opslagvraag van de applicatie. — **StorageClass**: recept om een volume te maken (*provisioner*, parameters, beleid). — **dynamische provisionering**: aanmaak van de PV op aanvraag. — `WaitForFirstConsumer`: aanmaak van het volume bij de eerste Pod. — `reclaimPolicy`: `Delete` of `Retain` bij het verwijderen van de PVC. — **toegangsmodi**: `ReadWriteOnce`, `ReadOnlyMany`, `ReadWriteMany`. — **CSI**: interface van de opslagdrivers. — **StatefulSet**: controller met stabiele identiteit en één volume per Pod. — `volumeClaimTemplates`: PVC-model per Pod. — **headless Service**: `clusterIP: None`, één DNS-naam per Pod. — **operator**: gespecialiseerde controller die een stateful applicatie uitbaat.